CVM++ Architecture and Workflow Guide

End-to-End Execution - Every Stage, Data Structure, Opcode, and Control Flow

CVM++ Documentation

May 2026

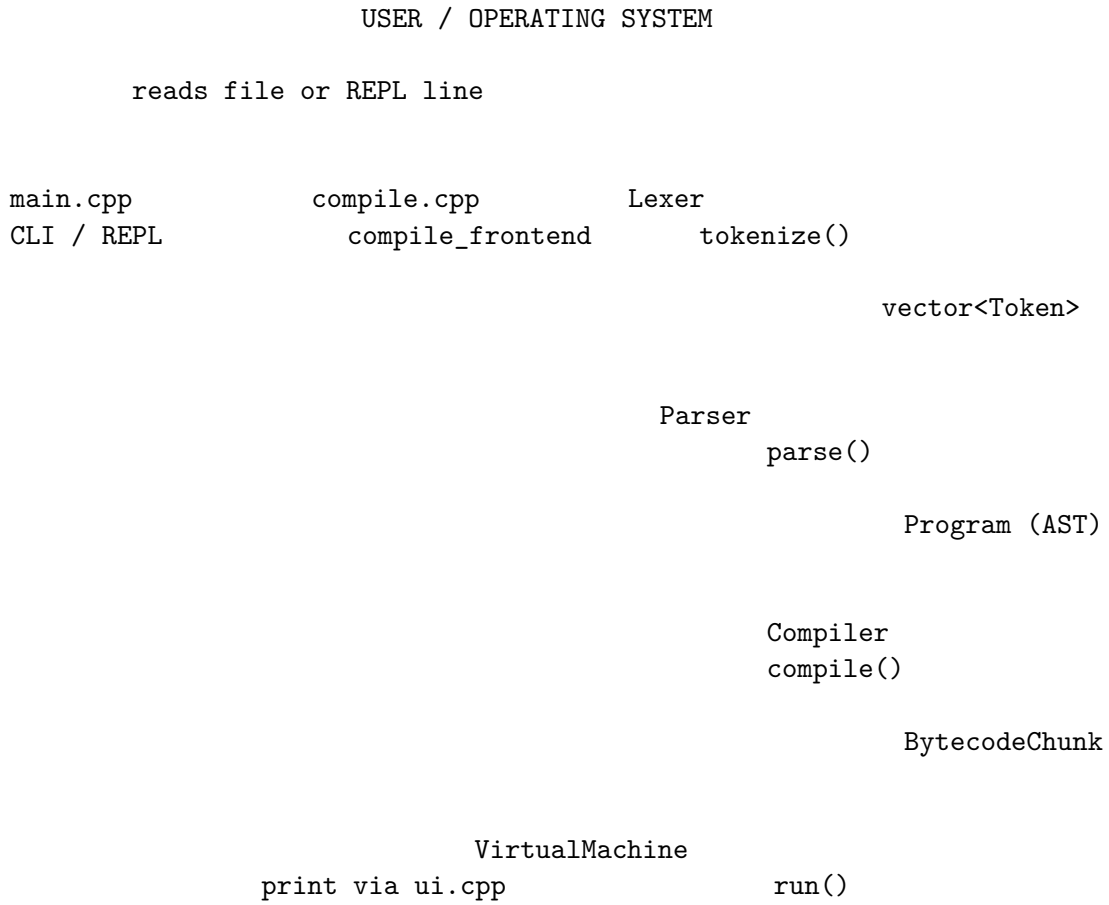
Contents

1 About This Document

This guide explains **how CVM++ works internally**: what happens from the moment you type `./build/cvmpp examples/hello.cvm` until output appears. For setup and definitions, see the **Project & Build Guide** (companion PDF).

---

2 1. Master Pipeline



output lines

stdout

Single orchestration function:

```
// src/compile.cpp
FrontEndResult compile_frontend(std::string source);
// 1. Lexer::tokenize()
// 2. Parser::parse()
// 3. Compiler::compile()
// Returns without running VM - main.cpp calls execute() separately.
```

---

3 2. Entry Points (`main.cpp`)

Mode	How triggered	What runs
File	`cvmpp script.cvm`	Read file → `compile_frontend` → `execute`
REPL	`cvmpp` (no args)	Loop: read line → `run_source` → optional `VmSession`
Compile-only	`cvmpp -c script.cvm`	Stops after bytecode; no VM
Debug	`cvmpp -d script.cvm`	Prints tokens, AST, bytecode tables
Quiet	`cvmpp -q script.cvm`	Suppresses status banners; prints program output only

REPL commands (`:help`, `:run`, `:disasm`, …) are handled before source is parsed.

---

4 3. Stage 1 — Lexer

4.1 3.1 Input / output

Input	`std::string` source (entire file or REPL buffer)
Output	`LexResult`: `vector<Token>` + `DiagnosticBag`
Files	`src/lexer.cpp`, `include/cvm++/lexer.hpp`

4.2 3.2 Algorithm (summary)

1. Scan left-to-right with `start_` / `current_` pointers.
2. Skip whitespace and `//` comments.
3. On digit → read integer (check overflow → error token).
4. On letter → identifier or keyword lookup (`let`, `fn`, `while`, …).

5. On `=`, `!`, `<`, `>` → single or double-char operators (`==`, `<=`, …).

6. Emit `Eof` at end.

4.3 3.3 Token types (what the parser sees)

Category	Examples
Literals	`Integer`, `True`, `False`
Keywords	`Let`, `Fn`, `Return`, `If`, `Else`, `While`, `Print`, `Input`
Identifiers	variable/function names
Operators	`Plus`, `EqualEqual`, `LessEqual`, …
Punctuation	`LParen`, `LBrace`, `Semicolon`, `Comma`
Sentinel	`Eof`, `Invalid`

4.4 3.4 What can fail

- Invalid character (`@`)
- Integer overflow
- Null byte in source

Phase tag: `LEXER`

---

5 4. Stage 2 — Parser

5.1 4.1 Input / output

Input	`vector<Token>` + original source string (for snippets)
Output	`ParseResult`: `unique_ptr<Program>` + diagnostics
Files	`src/parser.cpp`, `include/cvm++/parser.hpp`, `ast.hpp`

5.2 4.2 Program structure

```
struct Program {
    vector<unique_ptr<FunctionDecl>> functions;  // top-level fn defs
    vector<StmtPtr> statements;                  // "main" script body
};
```

Functions are parsed **before** main statements. Calls must refer to functions defined in the same file.

5.3 4.3 Grammar layers (precedence)

Level	Parses
`program`	functions* statements*
`statement`	let, assign, print, return, if, while, block, expr-stmt
`expression`	assignment (none at expr level — done in stmt)
`equality`	`== !=`

Level	Parses
`comparison`	`< > <= >=`
`term`	`+ -`
`factor`	`* /`
`unary`	`-`
`primary`	literal, identifier, call `f()`, `input`, `( expr )`

Technique: Recursive descent — each rule is a C++ function (`expression()`, `term()`, …).

5.4 4.4 AST node reference

Node	Fields (conceptual)	Meaning
`IntLiteralExpr`	`value`	Integer constant
`BoolLiteralExpr`	`value`	Boolean constant
`VariableExpr`	`name`	Load variable
`BinaryExpr`	`op`, `left`, `right`	Binary operation
`UnaryExpr`	`op`, `operand`	Negation
`CallExpr`	`callee`, `arguments`	Function call
`InputExpr`	—	Read stdin
`LetStmt`	`name`, `initializer`	First binding
`AssignStmt`	`name`, `value`	Reassignment
`PrintStmt`	`expression`	Output
`ReturnStmt`	`value`	Function return
`IfStmt`	`condition`, branches	Conditional
`WhileStmt`	`condition`, `body`	Loop
`BlockStmt`	`statements`	`{ ... }`
`FunctionDecl`	`name`, `parameters`, `body`	`fn` definition

5.5 4.5 What can fail

- Unexpected token, missing `;`, bad assignment target (`10 = x`)
- Unbalanced braces/parens

Phase tag: `PARSER`
Recovery: `synchronize()` skips to next `;` after error.

---

6 5. Stage 3 — Compiler

6.1 5.1 Input / output

Input	`Program` AST
Output	`CompileResult` with `BytecodeChunk`
Files	`src/compiler.cpp`, `bytecode.hpp`, `opcode.hpp`

6.2 5.2 Bytecode layout (critical)

```
Offset 0:   JUMP
           [function bodies]      skip
```

```
main entry
[top-level statements]
HALT
```

Without the initial `JUMP`, the VM would start inside a function body and crash on `LOAD_LOCAL`.

6.3 5.3 Name interning

Variable names are stored once in `chunk.names`. Instructions use **16-bit indices**:

- `LOAD_VAR index` — push global
- `STORE_VAR index` — pop into global

Inside functions, **local slots** use 8-bit indices (`LOAD_LOCAL`, `STORE_LOCAL`).

6.4 5.4 Control-flow codegen

`if (cond) { A } else { B }:`

1. Compile `cond`
2. `JUMP_IF_FALSE` → else (patch later)
3. Compile `A`
4. `JUMP` → end
5. Patch else label; compile `B`
6. Patch end label

`while (cond) { body }:`

1. Loop start label
2. Compile `cond`
3. `JUMP_IF_FALSE` → exit
4. Compile `body`
5. `JUMP` → loop start
6. Patch exit

6.5 5.5 Function codegen

1. Record `FunctionMeta { name, address, arity }` at current offset.
2. `begin_locals()` — map parameters to slots 0..arity-1.
3. Compile body block.
4. `end_locals()`.

Call site: compile arguments left-to-right, then `CALL fn_index argc`.

6.6 5.6 What can fail

- Undefined function call
- Wrong argument count
- Too many arguments (>255)

Phase tag: `COMPILER`

---

7 6. Stage 4 — Virtual Machine

7.1 6.1 Input / output

Input	`BytecodeChunk`, optional `VmSession*`, `istream` for `input`
Output	`VmResult`: diagnostics + captured print lines
Files	`src/vm.cpp`, `vm.hpp`, `value.hpp`

7.2 6.2 Runtime state

Field	Purpose
`ip_`	Instruction pointer into `chunk.code`
`stack_`	Operand stack (`vector<VmValue>`)
`globals_` + `global_init_`	File-mode variables by index
`frames_`	Call stack of `CallFrame`
`session_`	Optional REPL name → value map

7.3 6.3 Execution loop

```
steps = 0
while ip in bounds and no error:
    steps++
    if steps > 1_000_000: error "infinite loop"
    opcode = read_u8()
    execute_opcode(opcode)
```

7.4 6.4 Opcode reference (stack effects)

Notation: `a, b` = pop order (b top). `→` pushes.

Opcode	Operands	Effect
`PUSH_INT`	i64	→ int
`PUSH_BOOL`	u8	→ bool
`POP`	—	discard top
`LOAD_VAR`	u16 idx	→ global
`STORE_VAR`	u16 idx	pop → global
`LOAD_LOCAL`	u8 slot	→ local
`STORE_LOCAL`	u8 slot	pop → local
`ADD…DIV`	—	b, a → result

Opcode	Operands	Effect
`EQ…GE`	—	b, a → bool
`NEG`	—	a → -a
`INPUT`	—	→ value from stdin
`PRINT`	—	pop, append to output
`JUMP`	u32 off	ip = off
`JUMP_IF_FALSE`	u32 off	pop; if false, ip = off
`CALL`	u16 fn, u8 argc	pop argc args; push frame; ip = fn entry
`RETURN`	—	pop return value; restore frame; push value
`HALT`	—	stop

7.5 6.5 CALL / RETURN workflow

CALL:

1. Verify function index and stack has `argc` values.
2. Pop arguments into `CallFrame.locals`.
3. Save `return_ip = current ip`.
4. Set `ip = functions[fn].address`.

RETURN:

1. Pop return value from stack.
2. Restore `ip` from top frame; pop frame.
3. Push return value for caller.

7.6 6.6 Failsafes

Check	Error message style
Stack size > 65536	stack overflow
Pop empty stack	stack underflow
Division by zero	divide by zero
Uninitialized global	read of uninitialized variable
`RETURN` outside call	return outside function
`LOAD_LOCAL` outside call	LOAD_LOCAL outside function
Bad IP	IP overrun

Phase tag: `VM`

---

8 7. Stage 5 — UI and Diagnostics

Files: `ui.cpp`, `diagnostic.cpp`

Function	Role
`print_diagnostics`	Phase-colored errors with source caret
`print_token_table`	Debug lexer output
`print_ast_tree`	Indented AST
`print_bytecode_table`	Disassembly listing
`print_runtime_output`	Program `print` results

---

9 8. Worked Trace A — `print 1 + 2;`

Source:

```
print 1 + 2;
```

9.0.1 Lexer (tokens)

```
Print | Integer(1) | Plus | Integer(2) | Semicolon | Eof
```

9.0.2 Parser (AST sketch)

```
PrintStmt
    Binary(+)
         IntLiteral(1)
         IntLiteral(2)
```

9.0.3 Bytecode (conceptual)

```
0: PUSH_INT 1
9: PUSH_INT 2
18: ADD
19: PRINT
20: HALT
```

9.0.4 VM steps

Step	Opcode	Stack after
1	PUSH_INT 1	[1]
2	PUSH_INT 2	[1,2]
3	ADD	[3]
4	PRINT	[] (output: “3”)
5	HALT	done

---

10 9. Worked Trace B — `fn f(n) { return n + 1; } print f(4);`

10.0.1 Bytecode layout

```
0: JUMP → main
[ f body at address X ]
main:
```

```
... compile call f(4) ...
CALL f, 1
PRINT
HALT
```

10.0.2 CALL execution

1. Push argument `4`.
2. `CALL` → frame.locals[0]=4, ip → f entry.
3. `LOAD_LOCAL 0`, `PUSH_INT 1`, `ADD`, `RETURN` → stack gets `5`, ip → after CALL.
4. `PRINT` → output `5`.

---

11 10. REPL vs File Execution

Aspect	File mode	REPL mode
`VmSession`	`nullptr`	`&g_repl_session`
Globals storage	Indexed array in VM	`unordered_map` by name
State between runs	Reset each `run`	Persists across lines
Multiline	N/A (use `{ }` in one buffer)	Brace-depth prompt

---

12 11. File Dependency Graph

```
main.cpp
  → compile.hpp → lexer, parser, compiler, vm
  → ui.hpp

compile.cpp
  → lexer → token, diagnostic, source_loc
  → parser → ast
  → compiler → bytecode, opcode, ast
  → vm → bytecode, value, diagnostic

compiler.cpp → bytecode, opcode, ast, diagnostic
vm.cpp → bytecode, value, diagnostic
parser.cpp → ast, token, diagnostic
lexer.cpp → token, diagnostic
```

---

13 12. Debug Workflow (Recommended)

```
./build/cvmpp -d examples/functions.cvm
```

Study in order:

1. **Token table** — lexer classification
2. **AST tree** — parser structure
3. **Bytecode table** — compiler output
4. **Runtime output** — VM result

For bytecode-only:

```
./build/cvmpp
cvm++> :disasm examples/hello.cvm
```

---

14 13. CI Pipeline (GitHub Actions)

```
# .github/workflows/ci.yml
make          # build cvmpp
make verify   # run scripts/verify.sh on all examples
```

Ensures every public commit builds and passes integration scripts.

---

15 14. Extension Points (Future Work)

Feature	Touch modules
Strings	lexer, ast, value, vm, compiler
`for` loops	parser, compiler (desugar to while)
More types	value, vm opcodes, parser
Optimizer	new pass between AST and bytecode

CVM++ Architecture & Workflow Guide — companion to Project & Build Guide.